

# Big Data Mining



HANOI UNIVERSITY  
*HANU brings you to the world*

## Lecture 5: Association Rules Mining

Spring 2026

MSc. Bui Quoc Khanh

Faculty of Information Technology - HANU

# Sources

- Data Mining, The Textbook (2015) by Charu Aggarwal (Chapters 4, 5) – [slides by Lijun Zhang](#)
- Mining of Massive Datasets 2<sup>nd</sup> edition (2014) by Leskovec et al. ([Chapter 6](#)) - [slides](#)
- Data Mining Concepts and Techniques, 3<sup>rd</sup> edition (2011) by Han et al. (Chapter 6)
- Introduction to Data Mining 2<sup>nd</sup> edition (2019) by Tan et al. (Chapters 5, 6) – [slides ch5](#), [slides ch6](#)

# Association rule

- Let  $X, Y$  be two itemsets; the rule  $X \Rightarrow Y$  is an **association rule** of minimum support **minsup** and minimum confidence **minconf** if:

$$\text{sup}(X \cup Y) \geq \text{minsup}$$

and

$$\text{conf}(X \Rightarrow Y) \geq \text{minconf}$$

# Association rules mining has two main phases

- **In the first phase**, all the frequent itemsets are generated at the minimum support of **minsup**
  - This is a computationally expensive step
- **In the second phase**, the association rules are generated from the frequent itemsets at the minimum confidence level of **minconf**
  - This is relatively straightforward

# A straightforward implementation of the second phase

- For each frequent itemset  $I$  (i.e.,  $\text{sup}(I) \geq \text{minsup}$ )
  - For each possible partition  $X$ ,  $Y = I - X$ 
    - Check if  $\text{conf}(X \Rightarrow Y) \geq \text{minconf}$
- Use the **confidence monotonicity property** (next slide) to reduce search space

# Confidence monotonicity property

Let  $X_S, X_L, I$  be itemsets; assume  $X_S \subset X_L \subset I$

Then:

$$\text{conf}(X_L \Rightarrow I - X_L) \geq \text{conf}(X_S \Rightarrow I - X_S)$$

# Exercise: prove conf. monotonicity

$$X_S \subset X_L \subset I \Rightarrow \underline{\text{conf}(X_L \Rightarrow I - X_L) \geq \text{conf}(X_S \Rightarrow I - X_S)}$$


Tip: start from what you want to prove:

1. Use the definition of confidence on this

$$\text{conf}(X \Rightarrow Y) = \frac{\sup(X \cup Y)}{\sup(X)}$$

2. Try to arrive to  $\sup(X_L) \leq \sup(X_S)$

which we know is true because  $X_S \subset X_L$

# Brute-force itemset mining algorithms



# Naïve approach

- Generate all candidate itemsets ( $2^{|U|}$  of them)

Calculate  $\text{sup}(I)$  for every itemset

Not practical,  $U=1000 \Rightarrow$  more than  $10^{300}$  itemsets

- Key observation:

If there are no frequent  $k$ -itemsets,  
then there are no frequent  $(k+1)$ -itemsets

# Improved approach

Start with  $k=1$

Repeat:

- For each  $k$ -itemset
  - Determine  $\text{sup}(I)$
  - Keep if  $\text{sup}(I) \geq \text{minsup}$
- If no  $k$ -itemset has  $\text{sup}(I) \geq \text{minsup}$ , stop
- Otherwise,  $k \leftarrow k+1$  and repeat

# Improved approach is a significant improvement

- Let  $l$  be the final value of  $k$

$$\sum_{i=1}^l \binom{|U|}{i} \ll 2^{|U|}$$

- For  $|U| = 1000$ ,  $l=10$ , this is  $\simeq 10^{23}$

# Further improvements to brute-force method

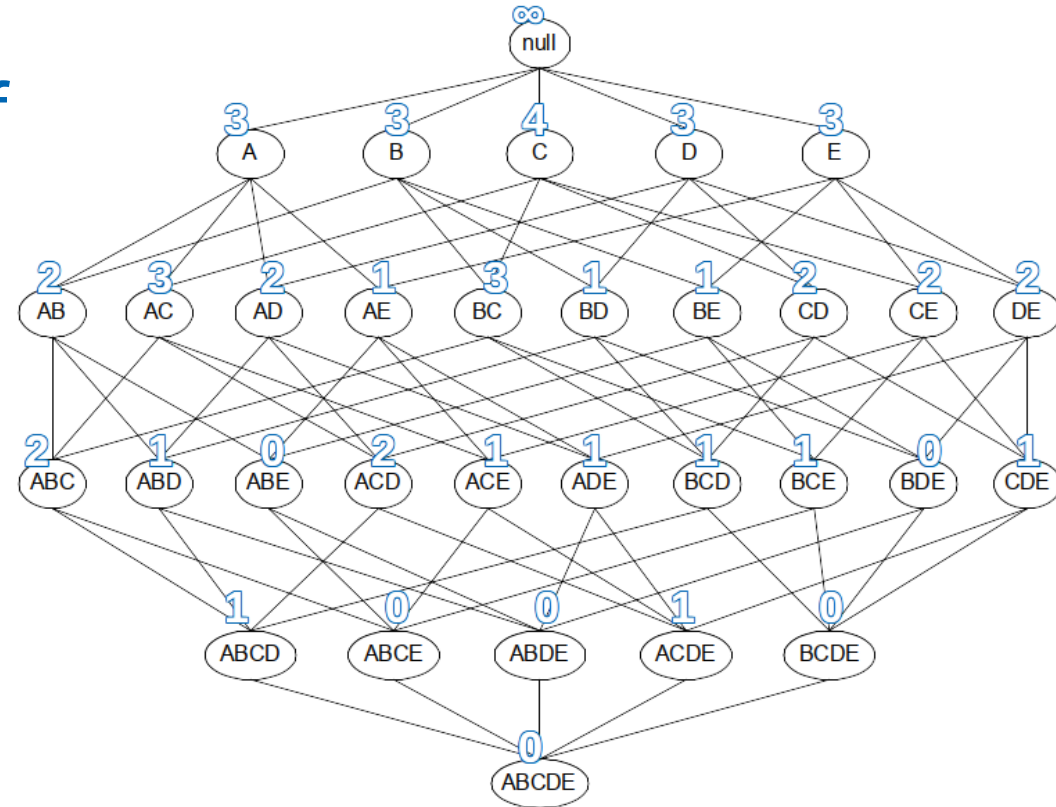
1. Reducing the size of the explored search space (lattice) by pruning candidate itemsets (lattice nodes) using tricks, such as the downward closure property
2. Counting the support of each candidate more efficiently by pruning transactions that are known to be irrelevant for counting a candidate itemset
3. Using compact data structures to represent either candidates or transaction databases that support efficient counting

# The Apriori Algorithm

[Rakesh Agrawal and Ramakrishnan Srikant, 1994]

# Apriori algorithm principle

- **Downward closure property:** every subset of a frequent itemset is also frequent
- Conversely, if an itemset has a subset that is not frequent, the itemset cannot be frequent
- What are subsets in the lattice?



# Example

| <i>TID</i> | <i>Items</i>              |
|------------|---------------------------|
| 1          | Bread, Milk               |
| 2          | Beer, Bread, Diaper, Eggs |
| 3          | Beer, Coke, Diaper, Milk  |
| 4          | Beer, Bread, Diaper, Milk |
| 5          | Bread, Coke, Diaper, Milk |



Items (1-itemsets)

| Item   | Count |   |
|--------|-------|---|
| Bread  | 4     |   |
| Coke   | 2     | X |
| Milk   | 4     |   |
| Beer   | 3     |   |
| Diaper | 4     |   |
| Eggs   | 1     | X |

support  $\geq 3/5$

# Example

| <i>TID</i> | <i>Items</i>              |
|------------|---------------------------|
| 1          | Bread, Milk               |
| 2          | Beer, Bread, Diaper, Eggs |
| 3          | Beer, Coke, Diaper, Milk  |
| 4          | Beer, Bread, Diaper, Milk |
| 5          | Bread, Coke, Diaper, Milk |



Items (1-itemsets)

| Item            | Count        |   |
|-----------------|--------------|---|
| Bread           | 4            |   |
| <del>Coke</del> | <del>2</del> | X |
| Milk            | 4            |   |
| Beer            | 3            |   |
| Diaper          | 4            |   |
| <del>Eggs</del> | <del>1</del> | X |

support  $\geq 3/5$



# Example (cont.)

Items (1-itemsets)

| Item   | Count |   |
|--------|-------|---|
| Bread  | 4     |   |
| Coke   | 2     | X |
| Milk   | 4     |   |
| Beer   | 3     |   |
| Diaper | 4     |   |
| Eggs   | 1     | X |

Pairs (2-itemsets)

| Item            | Count |   |
|-----------------|-------|---|
| {Bread, Milk}   | 3     |   |
| {Beer, Bread}   | 2     | X |
| {Bread, Diaper} | 3     |   |
| {Beer, Milk}    | 2     | X |
| {Diaper, Milk}  | 3     |   |
| {Beer, Diaper}  | 3     |   |

support  $\geq 3/5$

| TID | Items                     |
|-----|---------------------------|
| 1   | Bread, Milk               |
| 2   | Beer, Bread, Diaper, Eggs |
| 3   | Beer, Coke, Diaper, Milk  |
| 4   | Beer, Bread, Diaper, Milk |
| 5   | Bread, Coke, Diaper, Milk |

# Example (cont.)

Items (1-itemsets)

| Item            | Count        |              |
|-----------------|--------------|--------------|
| Bread           | 4            |              |
| <del>Coke</del> | <del>2</del> | <del>X</del> |
| Milk            | 4            |              |
| Beer            | 3            |              |
| Diaper          | 4            |              |
| <del>Eggs</del> | <del>1</del> | <del>X</del> |

Pairs (2-itemsets)

| Item                     | Count        |              |
|--------------------------|--------------|--------------|
| {Bread, Milk}            | 3            |              |
| <del>{Beer, Bread}</del> | <del>2</del> | <del>X</del> |
| {Bread, Diaper}          | 3            |              |
| <del>{Beer, Milk}</del>  | <del>2</del> | <del>X</del> |
| {Diaper, Milk}           | 3            |              |
| {Beer, Diaper}           | 3            |              |

| TID | Items                     |
|-----|---------------------------|
| 1   | Bread, Milk               |
| 2   | Beer, Bread, Diaper, Eggs |
| 3   | Beer, Coke, Diaper, Milk  |
| 4   | Beer, Bread, Diaper, Milk |
| 5   | Bread, Coke, Diaper, Milk |

support  $\geq 3/5$

# Example (cont.)

Items (1-itemsets)

| Item            | Count |   |
|-----------------|-------|---|
| Bread           | 4     |   |
| <del>Coke</del> | 2     | X |
| Milk            | 4     |   |
| Beer            | 3     |   |
| Diaper          | 4     |   |
| <del>Eggs</del> | 1     | X |

Pairs (2-itemsets)

| Item                     | Count |   |
|--------------------------|-------|---|
| {Bread, Milk}            | 3     |   |
| <del>{Beer, Bread}</del> | 2     | X |
| {Bread, Diaper}          | 3     |   |
| <del>{Beer, Milk}</del>  | 2     | X |
| {Diaper, Milk}           | 3     |   |
| {Beer, Diaper}           | 3     |   |

Triplets (3-itemsets)

| Item                  | Count |   |
|-----------------------|-------|---|
| {Bread, Diaper, Milk} | 2     | X |
| {Beer, Bread, Diaper} | 2     | X |
| {Bread, Diaper, Milk} | 2     | X |
| {Beer, Bread, Milk}   | 1     | X |

support  $\geq 3/5$

| TID | Items                     |
|-----|---------------------------|
| 1   | Bread, Milk               |
| 2   | Beer, Bread, Diaper, Eggs |
| 3   | Beer, Coke, Diaper, Milk  |
| 4   | Beer, Bread, Diaper, Milk |
| 5   | Bread, Coke, Diaper, Milk |

# Example (cont.)

Items (1-itemsets)

| Item            | Count        |              |
|-----------------|--------------|--------------|
| Bread           | 4            |              |
| <del>Coke</del> | <del>2</del> | <del>X</del> |
| Milk            | 4            |              |
| Beer            | 3            |              |
| Diaper          | 4            |              |
| <del>Eggs</del> | <del>1</del> | <del>X</del> |

Pairs (2-itemsets)

| Item                     | Count        |              |
|--------------------------|--------------|--------------|
| {Bread, Milk}            | 3            |              |
| <del>{Beer, Bread}</del> | <del>2</del> | <del>X</del> |
| {Bread, Diaper}          | 3            |              |
| <del>{Beer, Milk}</del>  | <del>2</del> | <del>X</del> |
| {Diaper, Milk}           | 3            |              |
| {Beer, Diaper}           | 3            |              |

Triplets (3-itemsets)

| Item                             | Count        |              |
|----------------------------------|--------------|--------------|
| <del>{Bread, Diaper, Milk}</del> | <del>2</del> | <del>X</del> |
| <del>{Beer, Bread, Diaper}</del> | <del>2</del> | <del>X</del> |
| <del>{Bread, Diaper, Milk}</del> | <del>2</del> | <del>X</del> |
| <del>{Beer, Bread, Milk}</del>   | <del>1</del> | <del>X</del> |

minsup 3/5, found 8 frequent itemsets

| TID | Items                     |
|-----|---------------------------|
| 1   | Bread, Milk               |
| 2   | Beer, Bread, Diaper, Eggs |
| 3   | Beer, Coke, Diaper, Milk  |
| 4   | Beer, Bread, Diaper, Milk |
| 5   | Bread, Coke, Diaper, Milk |

# Pseudocode of Apriori

**Algorithm** *Apriori*(Transactions:  $\mathcal{T}$ , Minimum Support: *minsup*)

**begin**

$k = 1$ ;

$\mathcal{F}_1 = \{ \text{All Frequent 1-itemsets} \}$ ;

**while**  $\mathcal{F}_k$  is not empty **do begin**

        Generate  $\mathcal{C}_{k+1}$  by joining itemset-pairs in  $\mathcal{F}_k$ ;

        Prune itemsets from  $\mathcal{C}_{k+1}$  that violate downward closure;

        Determine  $\mathcal{F}_{k+1}$  by support counting on  $(\mathcal{C}_{k+1}, \mathcal{T})$  and retaining  
        itemsets from  $\mathcal{C}_{k+1}$  with support at least *minsup*;

$k = k + 1$ ;

**end**;

**return**  $(\cup_{i=1}^k \mathcal{F}_i)$ ;

**end**

(1) Generation

(2) Pruning

(3) Support counting

# Exercise: Apriori

Get all rules of the form

$$\{a,b\} \rightarrow \{c\}$$

having:

support  $\geq 2/9$  and

confidence  $\geq 50\%$

*Note: to generate only rules of the form  $\{a,b\} \rightarrow \{c\}$ , consider only itemsets of size 3*

| TID | items       |
|-----|-------------|
| T1  | I1, I2 , I5 |
| T2  | I2,I4       |
| T3  | I2,I3       |
| T4  | I1,I2,I4    |
| T5  | I1,I3       |
| T6  | I2,I3       |
| T7  | I1,I3       |
| T8  | I1,I2,I3,I5 |
| T9  | I1,I2,I3    |

# Summary

# Things to remember

- **Support** and **confidence** on a rule
- **Downward closure** property
  - every subset of a frequent itemset is also frequent
  - hence, if an itemset  $X$  has a subset that is not frequent,  $X$  cannot be frequent
- **Apriori algorithm**



Speeding up the generation  
of candidate itemsets

# Speeding-up candidate generation: level-wise pruning trick

- Suppose we have examined itemsets up to size  $k$ 
  - Let  $F_k$  be the set of frequent  $k$ -itemsets [we know they are frequent]
  - Let  $C_{k+1}$  be the set of  $(k+1)$ -candidates [we do not know their frequency]
  - $I \in C_{k+1}$  is frequent only if all the  $k$ -subsets of  $I$  are frequent
- Pruning
  - Generate all the  $k$ -subsets of  $I$
  - If any one of them does not belong to  $F_k$ , then remove  $I$

# Candidates generation

- A **naïve approach**

Check all the possible combinations of frequent itemsets

- An example of the **naïve approach**

If frequent itemsets found no far: {abc} {bcd} {abd} {cde}

Then try the following:

- {abc} + {bcd} = {abcd}
- {bcd} + {abd} = {abcd}
- {abd} + {cde} = {abcde}
- ....

# Candidates generation (cont.)

- Introduction of **ordering**
  - Items in  $U$  can be sorted in lexicographic ordering
  - Items in each itemset can be sorted in lexicographic ordering
  - Itemsets can be ordered as strings
- The improved approach:
  - Order the frequent  $k$ -itemsets
  - Merge two itemsets if and only if the first  $k-1$  items of them are equal

# Candidates generation (cont.)

- Example 1:

*Note: We are writing  $\{xyz\}$  to mean the set  $\{x, y, z\}$*

- k-itemsets:  $\{abc\}$   $\{abd\}$   $\{acd\}$   $\{bcd\}$
- (k+1)-itemsets:  $\{abc\} + \{abd\} = \{abcd\}$
- No other pair shares a prefix of size k-1, no need to check other combinations

- Example 2:

- k-itemsets:  $\{abc\}$   $\{acd\}$   $\{bcd\}$
- We do not generate any (k+1)-itemsets as candidates
- **Did we miss  $\{abcd\}$ , could it have been frequent?**

# Candidates generation (cont.)

- Example 1:

*Note: We are writing  $\{xyz\}$  to mean the set  $\{x, y, z\}$*

- k-itemsets:  $\{abc\}$   $\{abd\}$   $\{acd\}$   $\{bcd\}$
- (k+1)-itemsets:  $\{abc\} + \{abd\} = \{abcd\}$
- No other pair shares a prefix of size k-1, no need to check other combinations

- Example 2:

- k-itemsets:  $\{abc\}$   $\{acd\}$   $\{bcd\}$
- We do not generate any (k+1)-itemsets as candidates
- Did we miss  $\{abcd\}$ , could it have been frequent?
  - **No**: due to the **Downward Closure Property** every subset of a frequent itemset is also frequent, and  $\{abd\}$  is not frequent, hence  $\{abcd\}$  cannot be frequent

Improving computation of support

# Naïve support counting

- **Naïve counting:**
  - For each candidate  $l_i \in C_{k+1}$ 
    - For each transaction  $T_j$  in  $T$ 
      - Check whether  $l_i$  appears in  $T_j$
- This is very slow if both  $|C_{k+1}|$  and  $|T|$  are large



# Support counting with a data structure

- A Better Approach
  - Organize the candidate patterns in  $C_{k+1}$  in a **data structure**
- **Use the data structure to accelerate counting**
  - Each transaction in  $T_i$  examined against the subset of candidates in  $C_{k+1}$  that *might* be contained in  $T_i$

# Support counting based on hashing

## Naïve counting:

For each  $l_i \in C_{k+1}$   
  For all  $T_j \in T$   
    If  $l_i \subseteq T_j$   
      Add to  $\text{sup}(l_i)$

## Hashed counting:

For each  $T_j \in T$   
  For  $l_i \in \text{hashbucket}(T_j, C_{k+1})$   
    If  $l_i \subseteq T_j$   
      Add to  $\text{sup}(l_i)$

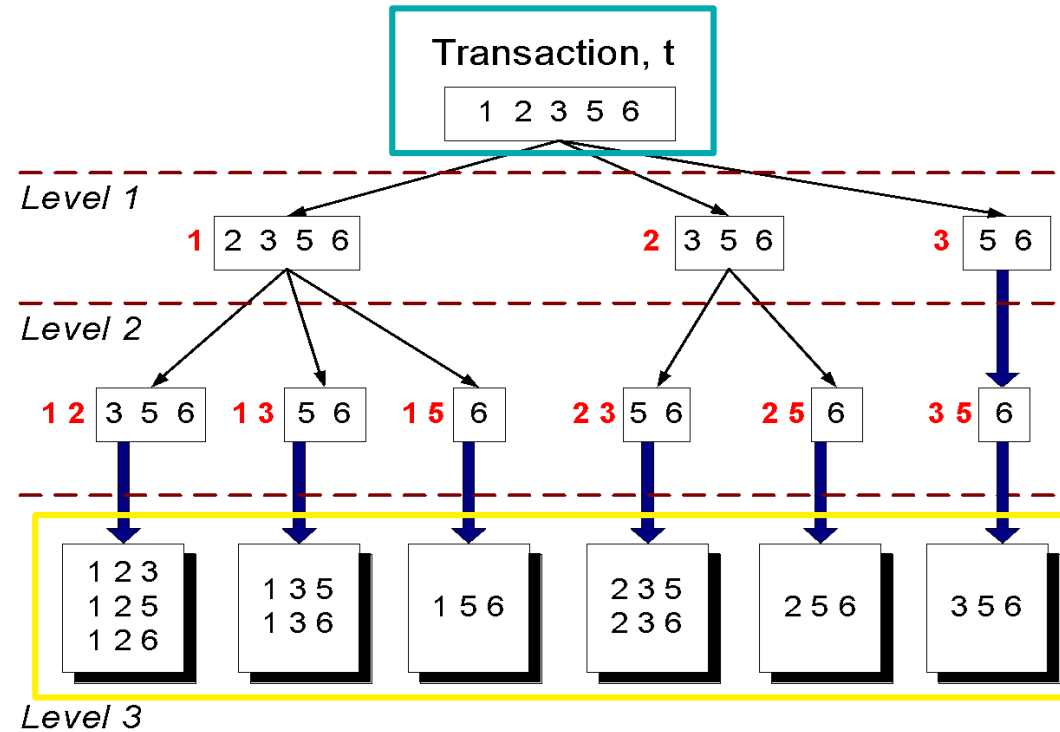
# Which candidates are relevant?

Imagine 15 candidate itemsets of length 3:

- {1 4 5}, {1 2 4}, {4 5 7},  
{1 2 5}, {4 5 8}, {1 5 9},  
{1 3 6}, {2 3 4}, {5 6 7},  
{3 4 5}, {3 5 6}, {3 5 7},  
{6 8 9}, {3 6 7}, {3 6 8}

Now, suppose we look for this transaction:

{1 2 3 5 6}



Here we depict only the candidates that appear in the transaction (10 out of 15)

# Hash tree for itemsets in $C_{k+1}$

- A tree with fixed degree  $r$
- Each itemset in  $C_{k+1}$  is stored in a leaf node
- All internal nodes use a hash function to map items to one of the  $r$  branches (can be the same for all internal nodes)
- All leaf nodes contain a lexicographically sorted list of up to `max_leaf_size` itemsets

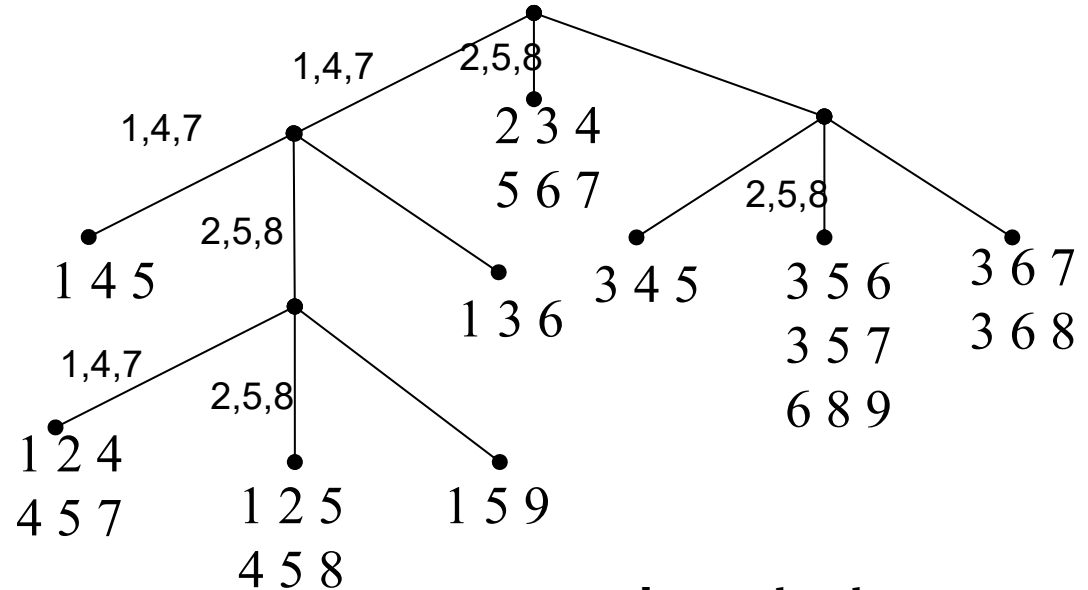
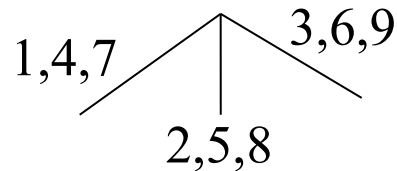
# Example hash tree

$r=3$     $\text{max\_leaf\_size}=3$

## Candidate itemsets

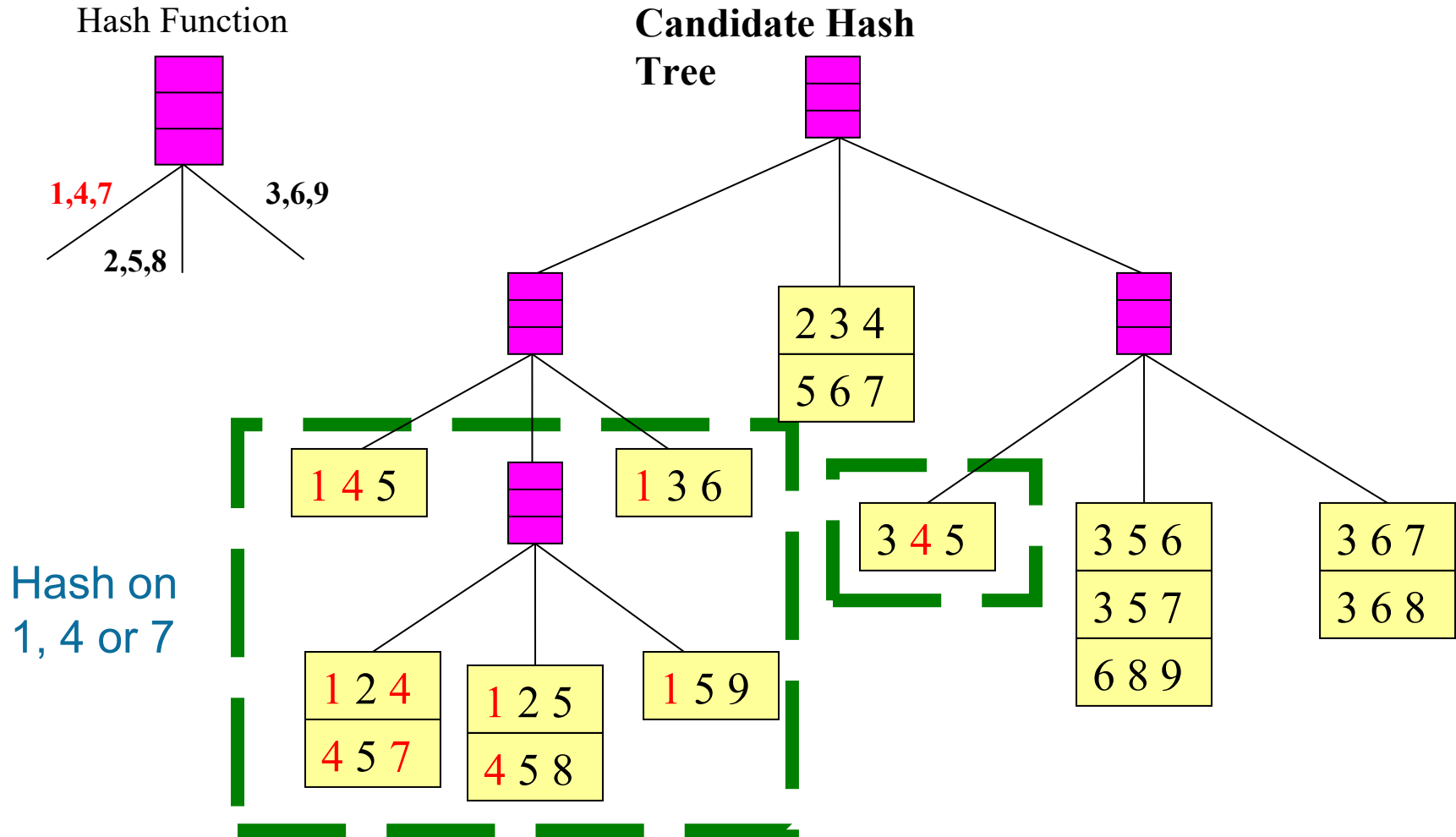
{1 4 5}, {1 2 4}, {4 5 7},  
{1 2 5}, {4 5 8}, {1 5 9},  
{1 3 6}, {2 3 4}, {5 6 7},  
{3 4 5}, {3 5 6}, {3 5 7},  
{6 8 9}, {3 6 7}, {3 6 8}

### Hash function

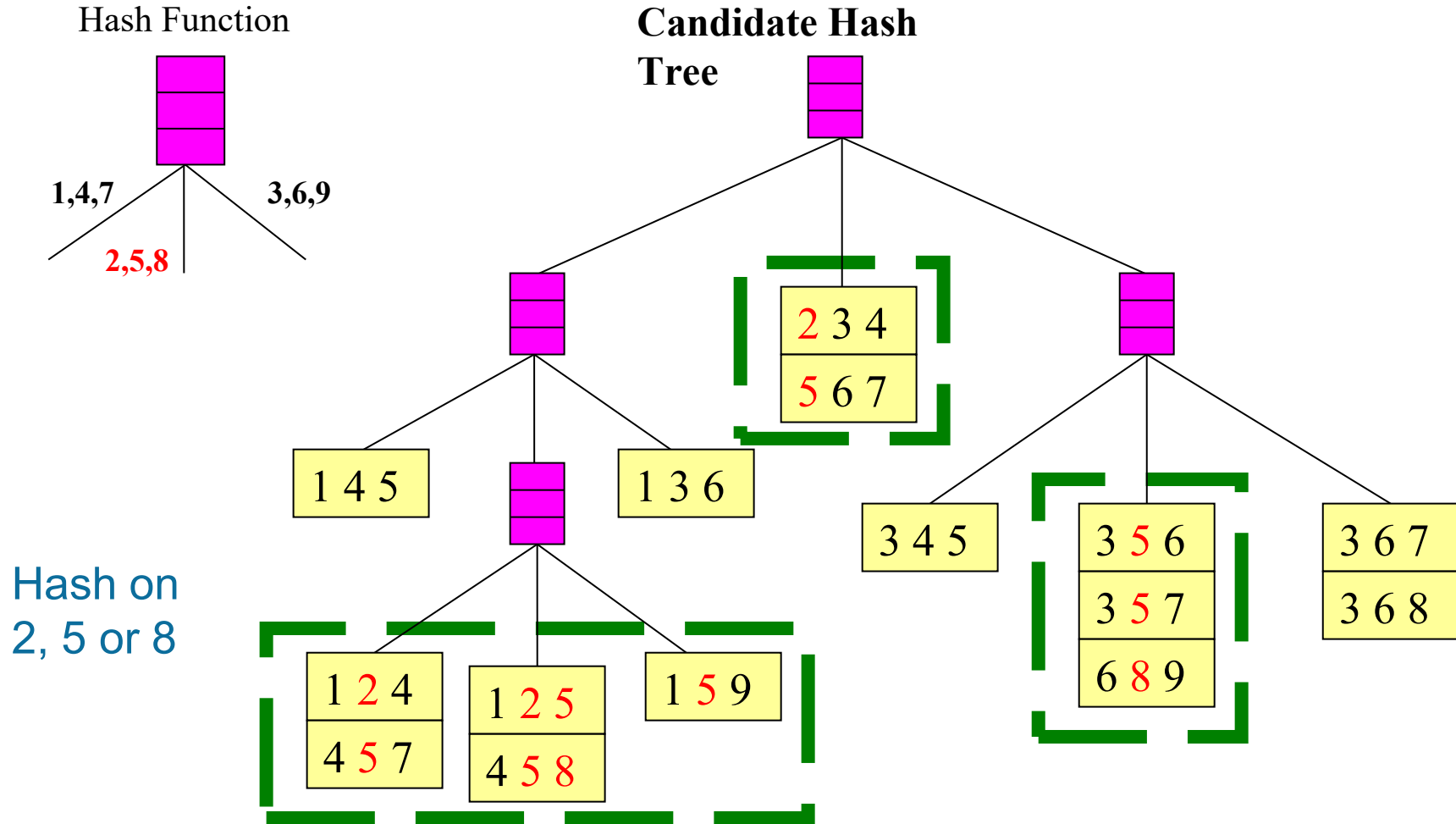


**Important:  
itemsets are sorted!**

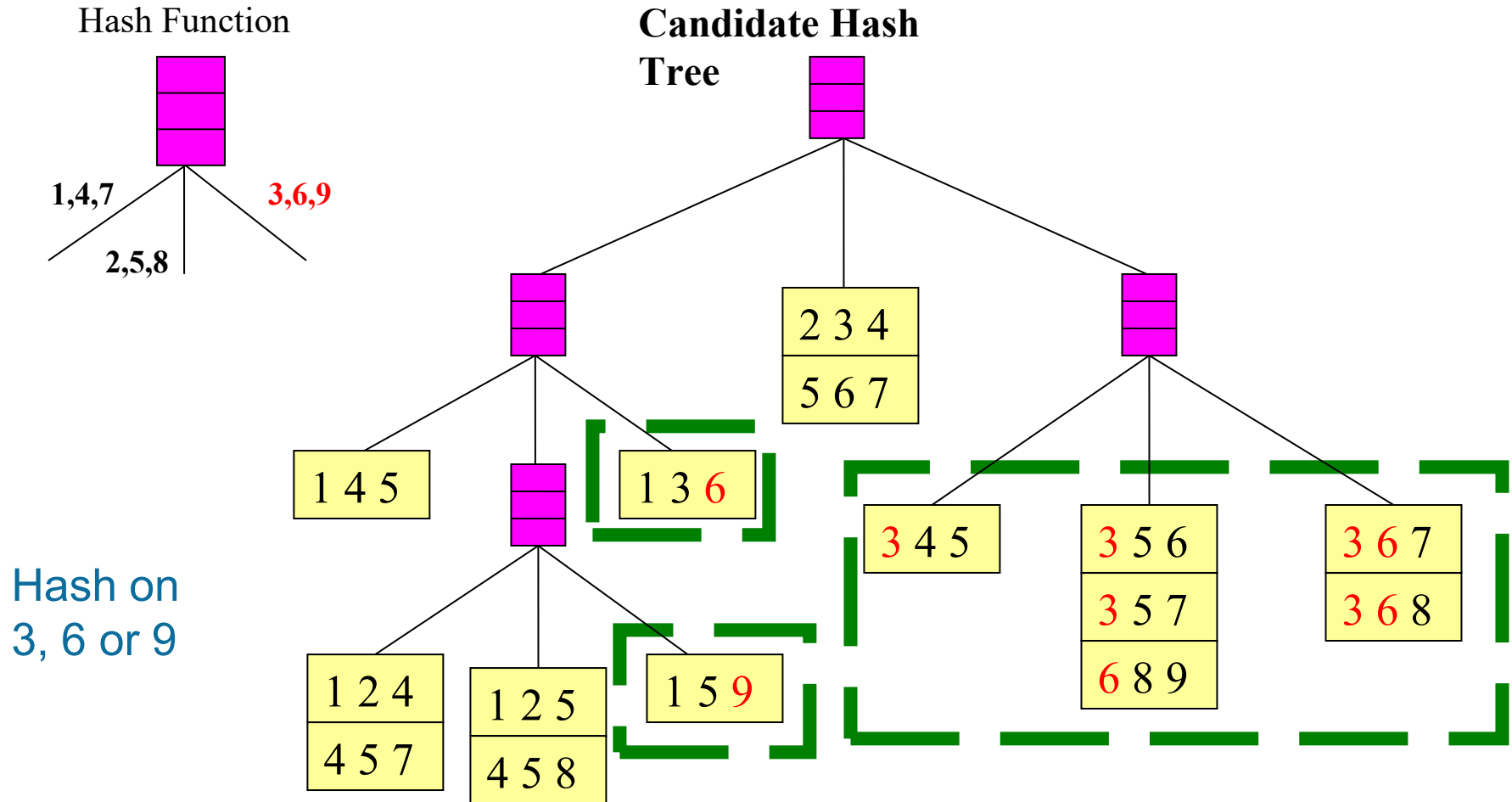
# Example hash tree (cont.)



# Example hash tree (cont.)

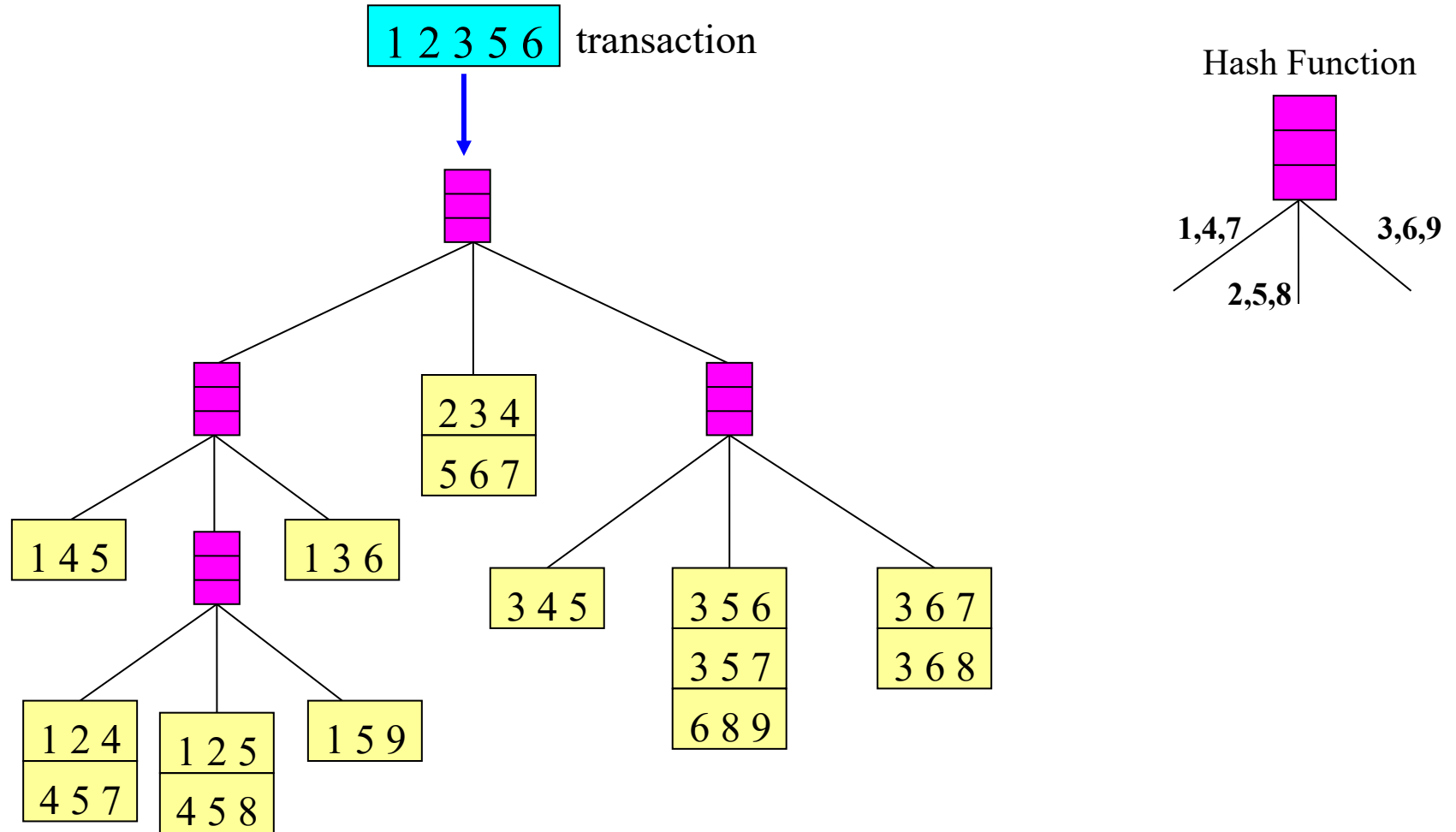


# Example hash tree (cont.)

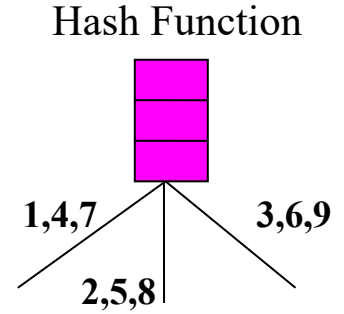
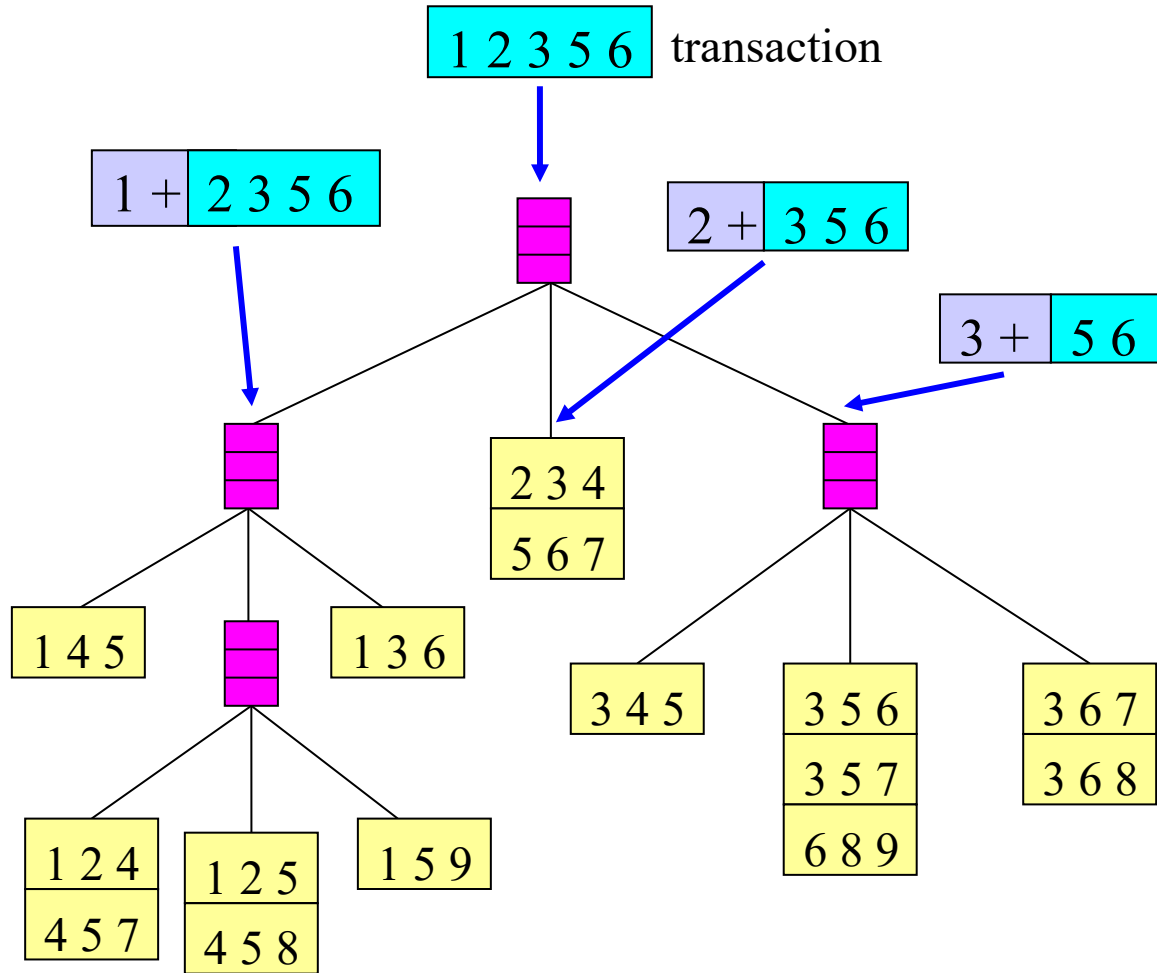




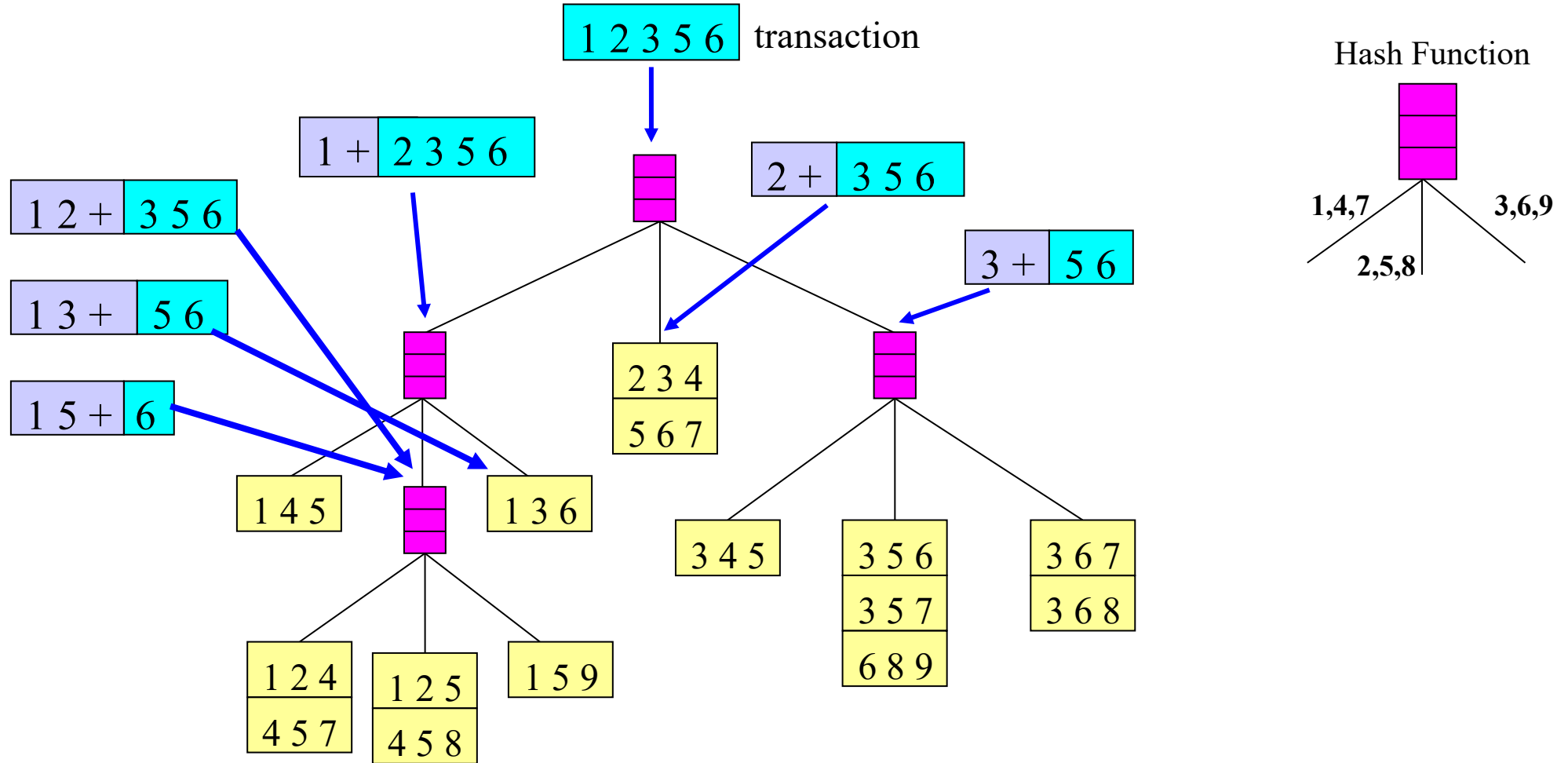
# Checking which candidates might be in a transaction



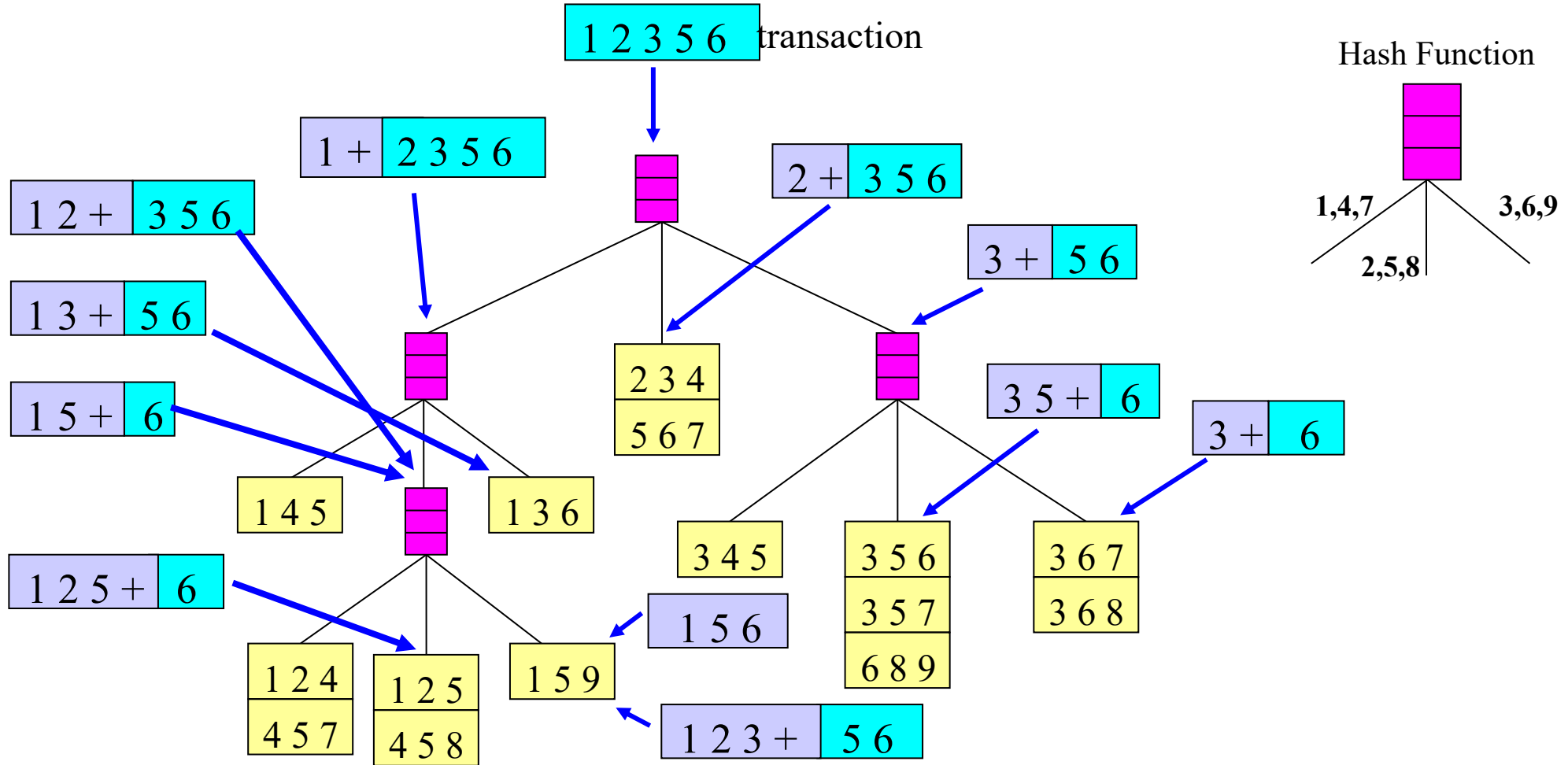
# Checking which candidates might be in a transaction



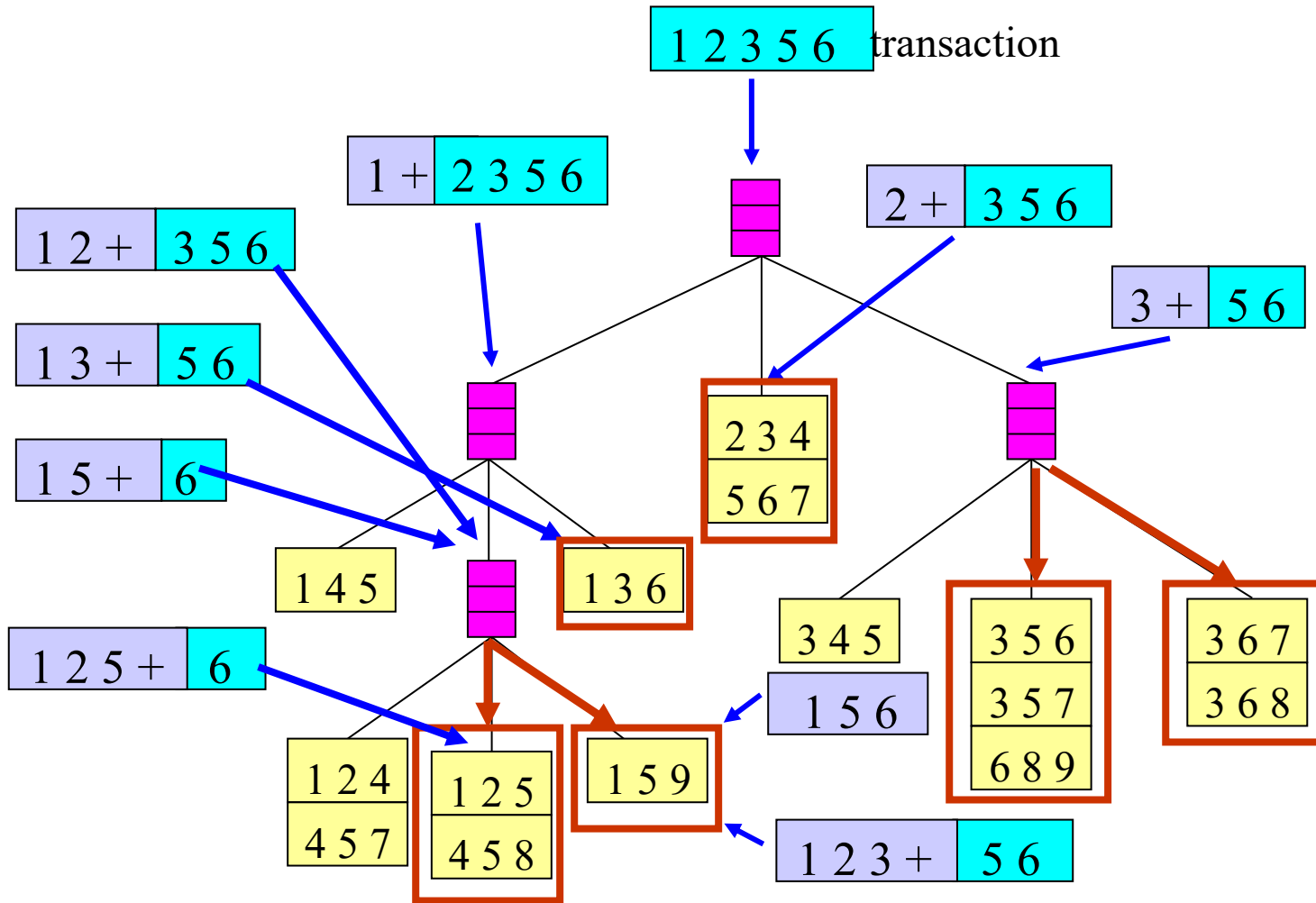
# Checking which candidates might be in a transaction



# Checking which candidates might be in a transaction



# Checking which candidates might be in a transaction



Hash Function

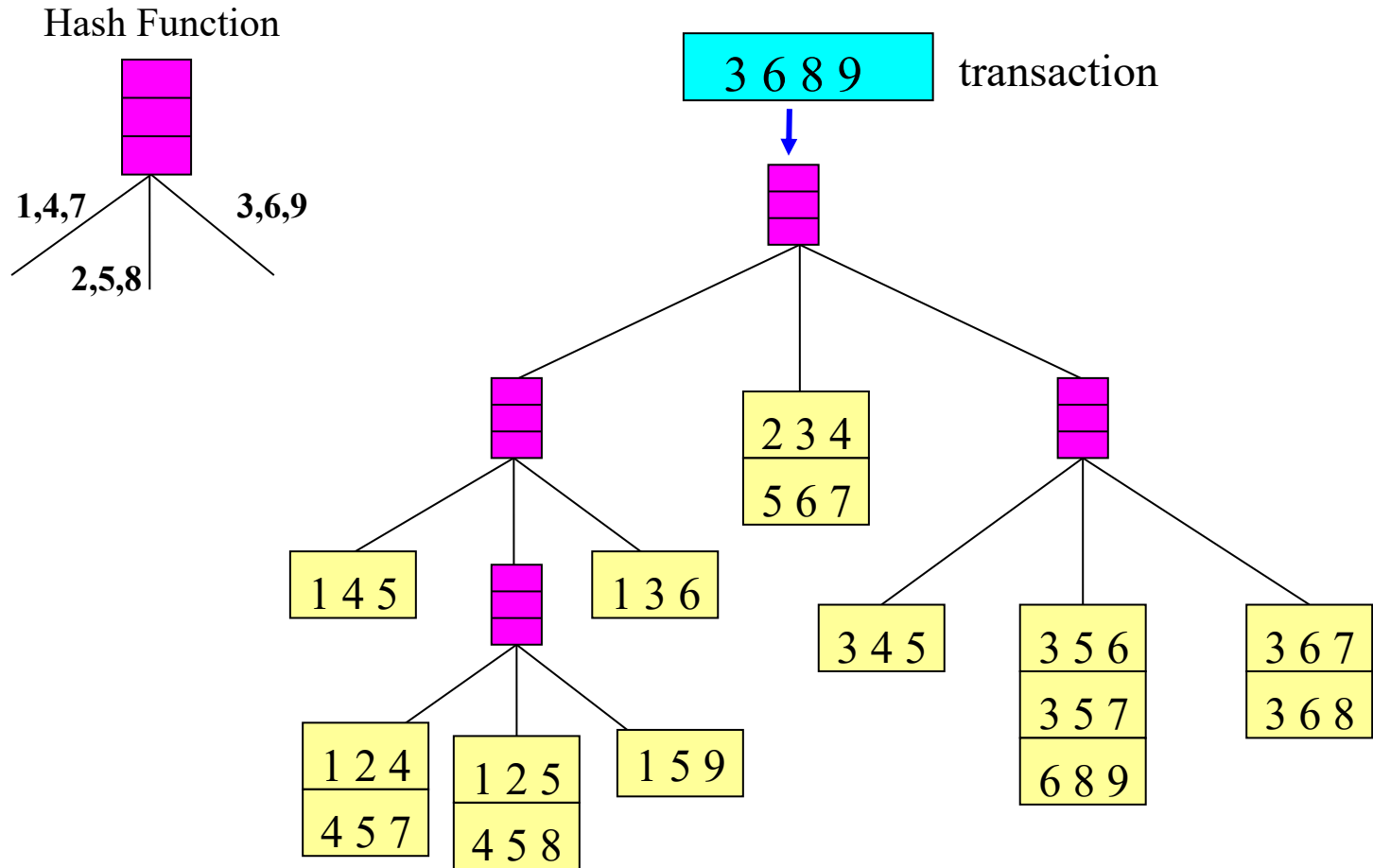
1,4,7

2,5,8

3,6,9

**Compare  
transaction against  
11 out of 15  
candidates**

# Exercise: Use the hash tree to determine which candidates might be in this transaction



# Summary: efficient frequent itemsets

- $C_1 \leftarrow$  singletons, lexicographically sorted
- $F_1 \leftarrow$  elements in  $C_1$  with support  $\geq$  minsup, obtained by direct counting
- $k \leftarrow 1$
- While  $F_k$  is not empty
  - Generate  $C_{k+1}$  by merging elements in  $F_k$  sharing a prefix of size  $k-1$
  - Remove from  $C_{k+1}$  elements that do not have all of their subsets in  $F_k$
  - Create hash tree for  $C_{k+1}$
  - Pass all transactions in  $T$  by the hash tree to compute support for elements of  $C_{k+1}$
  - $F_{k+1} \leftarrow$  elements in  $C_{k+1}$  with support  $\geq$  minsup, lexicographically sorted
- Return the union of  $F_1, F_2, \dots, F_k$

# Summary



# Things to remember

- Candidate generation and merging
  - Keep lexicographically ordered
  - Merge if they share a prefix
- Hash-tree method for counting transactions containing a set of candidate itemsets

# Exercises for this topic

- Data Mining, The Textbook (2015) by Charu Aggarwal
  - Exercises 4.9 → 9-10
- Mining of Massive Datasets 2<sup>nd</sup> edition (2014) by Leskovec et al.
  - Exercises 6.2.7 → 6.2.5 and 6.2.6
- Introduction to Data Mining 2<sup>nd</sup> edition (2019) by Tan et al.
  - Exercises 5.10 → 9-12